

Entity Recognition System

Comparative NER using spaCy and Hugging Face Transformers

Project Goal

Build an end-to-end Named Entity Recognition system that converts unstructured text into structured entities, compares a lightweight spaCy model with a transformer-based model, and selects the most practical model using accuracy, speed, model size, and reliability.

Prepared for: Portfolio, learning, and interview demonstration

Version 1.0 | June 2026

1. Project Overview

The Entity Recognition System will identify and classify important terms from raw text. The system will focus on business, technology, AI, and employment-related text and return structured entities that can be searched, analyzed, or passed to downstream applications.

1.1 Primary Objective

- Extract PERSON, ORGANIZATION, LOCATION, DATE, and DOMAIN_TERM entities from unstructured English text.
- Prepare and validate a custom BIO-tagged dataset.
- Train and evaluate both a spaCy NER model and a transformer token-classification model.
- Compare the two approaches using precision, recall, F1 score, inference speed, model size, and error patterns.
- Expose the selected model through a small API or web interface in a later phase.

1.2 Problem Statement

Most business text is unstructured. Job descriptions, company reports, emails, news, and technical documents contain useful names, organizations, locations, dates, and specialist terms, but ordinary software cannot directly use them. This project will transform that text into structured records.

1.3 Example

Input text	Expected entities
Ali Raza joined Systems Limited in Lahore on 15 June 2026 to work on machine learning.	Ali Raza → PERSON Systems Limited → ORGANIZATION Lahore → LOCATION 15 June 2026 → DATE machine learning → DOMAIN_TERM

2. Scope

2.1 In Scope

- English-language text processing.
- Custom BIO annotation and validation.
- Five entity categories and their B-/I- labels.
- spaCy training pipeline.
- Hugging Face transformer fine-tuning pipeline.
- Model evaluation and side-by-side comparison.
- Error analysis, especially organization-versus-location confusion.
- Rule-based post-processing where justified by evidence.
- A simple inference interface in the final phase.

2.2 Out of Scope for the First Version

- Multilingual NER.

- Handwriting or image-based text extraction.
- Large-scale distributed training.
- Automatic annotation without human review.
- Production-grade cloud deployment and monitoring.
- Personally identifying sensitive-data profiling.

3. Functional Requirements

ID	Requirement
FR-01	The system shall accept plain text as input.
FR-02	The system shall return entity text, entity type, start position, end position, and confidence where supported.
FR-03	The dataset validator shall detect invalid labels, missing fields, and broken BIO sequences.
FR-04	The training pipeline shall split data into train, validation, and test sets without sentence leakage.
FR-05	The project shall train or configure both spaCy and transformer-based NER models.
FR-06	The evaluator shall calculate per-class and overall precision, recall, and F1 score.
FR-07	The comparison report shall include inference time and model size.
FR-08	The error-analysis module shall store incorrect predictions with expected and predicted labels.

3.1 Non-Functional Requirements

- Maintainability: modular Python code with type hints, docstrings, configuration files, and tests.
- Reproducibility: fixed random seeds, recorded package versions, and saved experiment settings.
- Usability: simple commands for validation, training, evaluation, and inference.
- Performance: target interactive inference for short documents on a normal laptop.
- Reliability: invalid datasets must fail clearly instead of being silently corrected.
- Portability: training should work locally where possible and on Google Colab for transformer experiments.

4. Entity Schema and BIO Tagging

Entity	Meaning	Examples
--------	---------	----------

PERSON	Human names	Ali Raza, Sara Ahmed
ORGANIZATION	Companies, institutions, teams	Systems Limited, Punjab University
LOCATION	Cities, countries, regions, physical places	Lahore, Pakistan, Johar Town
DATE	Calendar dates and meaningful date expressions	15 June 2026, Monday, last week
DOMAIN_TERM	Technical or domain-specific concepts	machine learning, REST API, named entity recognition

4.1 BIO Rules

- B-ENTITY marks the first token of an entity.
- I-ENTITY marks the following token or tokens of the same entity.
- O marks a token that is outside every entity.
- An I-tag cannot start a sentence and cannot follow a different entity type.

Example: “Systems Limited hired Ali Raza.” → Systems/B-ORGANIZATION, Limited/I-ORGANIZATION, hired/O, Ali/B-PERSON, Raza/I-PERSON, ./O

5. Proposed System Architecture

The project will use one shared dataset and evaluation pipeline so the two model families can be compared fairly.

1. Raw text collection
2. Manual annotation using BIO labels
3. Dataset validation and train/validation/test split
4A. spaCy training pipeline 4B. Transformer fine-tuning pipeline
5. Shared evaluation and benchmarking
6. Error analysis and optional rule-based corrections
7. Inference API / demonstration interface

5.1 Main Components

Component	Responsibility
Data Layer	Raw text, BIO CSV/JSON files, label definitions, validated splits.
Validation Layer	Schema checks, missing values, allowed labels,

	BIO consistency, duplicate detection.
spaCy Pipeline	Tokenizer, NER component, configuration, training, model packaging.
Transformer Pipeline	Tokenizer alignment, token classification model, Trainer loop, checkpoints.
Evaluation Layer	segeval metrics, speed benchmark, model-size comparison, confusion analysis.
Inference Layer	Unified output format regardless of the selected model.

6. Technology Stack

Area	Technology	Purpose
Language	Python	Core implementation and experimentation
Data handling	Pandas	Load, inspect, clean, and transform annotations
Classical NER	spaCy	Fast trainable NLP pipeline and rule integration
Transformer NER	Hugging Face Transformers	Fine-tune a pretrained token-classification model
Dataset utilities	Hugging Face Datasets	Dataset mapping and preprocessing
Metrics	segeval / evaluate	Entity-level precision, recall, and F1
Utilities	scikit-learn	Data splitting and supporting analysis
Testing	pytest	Unit and validation tests
Optional interface	FastAPI or Streamlit	Demonstrate inference in the final phase

7. Dataset Design

7.1 Initial Data Format

The first version will use one token per row:

sentence_id	token	label
-------------	-------	-------

1	Ali	B-PERSON
1	works	O
1	at	O
1	Google	B-ORGANIZATION

7.2 Data Quality Rules

- Every sentence must have a unique sentence identifier.
- Every token must have exactly one valid label.
- Punctuation should normally be stored as separate tokens.
- Entity spans must follow valid BIO transitions.
- Train, validation, and test sets must not share the same sentence.
- The test set must remain untouched until final evaluation.
- Annotation guidelines must define difficult cases such as company branches, abbreviations, and overlapping-looking terms.

7.3 Suggested Dataset Size

For a credible portfolio prototype, target approximately 1,000 to 2,000 manually reviewed sentences. Begin with 100 to 200 sentences to verify the full pipeline, then expand. The priority is consistent annotation rather than raw volume.

8. Model Strategy

8.1 spaCy Baseline

- Train a custom spaCy NER component using the shared entity schema.
- Use it as the lightweight baseline.
- Measure CPU inference speed and package size.
- Optionally combine predictions with carefully tested EntityRuler patterns.

8.2 Transformer Model

- Start with a compact pretrained model such as DistilBERT or a similar token-classification backbone.
- Align word-level BIO labels with subword tokens.
- Ignore special tokens and non-first subword pieces where required by the chosen alignment strategy.
- Fine-tune on the same train and validation data used for spaCy.
- Save the best checkpoint based on validation F1 score.

8.3 Fair Comparison

Both models must use the same entity definitions and the same held-out test set. Differences in tokenization must be normalized before entity-level evaluation.

9. Evaluation Plan

Metric	Why it matters	Preferred interpretation
Precision	How many predicted entities are correct	Higher precision means fewer false entities
Recall	How many real entities were found	Higher recall means fewer missed entities
F1 score	Balance between precision and recall	Primary quality metric
Per-class F1	Shows weak entity categories	Important for DOMAIN_TERM and DATE
Inference latency	Time per sentence/document	Important for interactive use
Model size	Storage and deployment cost	Important for laptop or API deployment
Error categories	Explains failure patterns	Guides annotation and rule improvements

9.1 Success Criteria for Version 1

- All dataset validation tests pass.
- Both models train successfully and produce repeatable evaluation results.
- Overall entity-level F1 score reaches at least 0.80 on a clean internal test set.
- The report clearly explains which model is recommended and why.
- The final demo returns structured entities from unseen text.

10. Error Analysis and Reliability

The project will not treat a single F1 score as the whole story. Incorrect predictions will be grouped into useful categories.

- Boundary error: only part of a multi-token entity is detected.
- Type error: the span is correct but the entity category is wrong.
- Missed entity: a true entity is labeled O.
- False positive: ordinary text is incorrectly marked as an entity.
- Ambiguity error: for example, “Amazon” as a company versus a geographic location.
- Domain-term inconsistency: technical phrases are annotated differently across samples.

Important rule: Manual error correction means improving annotations, adding representative training examples, or applying documented post-processing rules. It does not mean secretly editing model outputs during evaluation.

11. Development Phases

Phase	Deliverable
Phase 1: Foundation	Repository structure, label schema, sample BIO dataset, configuration, validator, tests, and README.
Phase 2: Data Preparation	Annotation guidelines, larger dataset, cleaning, splitting, and dataset statistics.
Phase 3: spaCy Baseline	Conversion to spaCy format, configuration, training, evaluation, and saved model.
Phase 4: Transformer Model	Token alignment, fine-tuning, checkpointing, and evaluation.
Phase 5: Comparison	Shared metrics, latency benchmark, model-size report, and recommendation.
Phase 6: Reliability	Error analysis, improved examples, optional rules, and rerun experiments.
Phase 7: Demonstration	Unified inference module and a small FastAPI or Streamlit interface.

12. Proposed Repository Structure

entity-recognition-system/

- ├─ data/
 - │ └─ raw/
 - │ └─ annotated/
 - │ └─ processed/
- ├─ src/
 - │ └─ data/
 - │ └─ spacy_model/
 - │ └─ transformer_model/
 - │ └─ evaluation/
 - │ └─ inference/
 - │ └─ config.py
- ├─ tests/
- ├─ models/
- ├─ reports/
- ├─ notebooks/
- ├─ requirements.txt
- ├─ README.md
- └─ .gitignore

13. Risks and Mitigations

Risk	Impact	Mitigation
Small dataset	Overfitting and unstable metrics	Start with a complete pipeline, then expand reviewed examples.
Inconsistent annotations	Model learns contradictory patterns	Write annotation guidelines and review samples regularly.
Class imbalance	Rare entities receive poor recall	Track per-class counts and add targeted examples.
Subword alignment errors	Wrong transformer labels	Add unit tests for tokenizer-to-label alignment.
Data leakage	Unrealistically high test results	Split by sentence or document before transformation.
Overuse of rules	Good demo but weak generalization	Use rules only after measured error analysis.

14. Immediate Next Milestone

The first implementation milestone is Phase 1: Foundation. Codex should create only the repository structure, label definitions, a small realistic BIO dataset, a strict validator, unit tests, and setup documentation. No model training should be added until the data contract is stable and all validation tests pass.

14.1 Definition of Done

- The project installs successfully in a clean Python environment.
- The sample dataset includes every entity class and valid multi-token entities.
- The validator reports dataset statistics and rejects invalid BIO sequences.
- All pytest tests pass.
- The README provides exact commands from setup through validation.
- No training, API, or user-interface code exists yet.